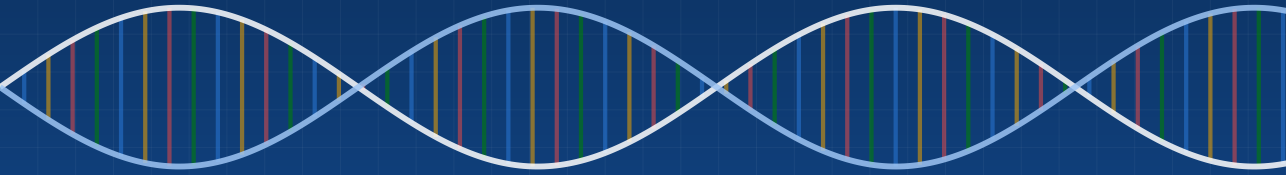


Alineamiento múltiple, motivos y perfiles



Los HMM de perfil convierten un alineamiento múltiple de secuencias en un sistema de puntuación específico de cada posición, adecuado para buscar en bases de datos secuencias remotamente homólogas.

— Eddy (1998)

● Al terminar este capítulo usted podrá

- Definir un alineamiento múltiple de secuencias (MSA), puntuarlo con la suma de pares y explicar por qué su solución exacta es computacionalmente intratable.
- Reproducir paso a paso el alineamiento progresivo: distancias, árbol guía y alineamiento de perfiles; reconocer sus limitaciones y cómo las corrigen MUSCLE, MAFFT, T-Coffee y Clustal Omega.
- Construir una matriz de peso posicional (PWM) a partir de sitios de unión, con pseudoconteos, y usarla para buscar sitios en una secuencia.
- Calcular la entropía y el contenido de información de cada posición de un motivo e interpretar un *sequence logo*.
- Formular un modelo oculto de Márkov y resolver sus tres problemas clásicos con los algoritmos *forward*, de Viterbi y de Baum-Welch.

- Describir la arquitectura de un HMM de perfil (estados de coincidencia, inserción y delección) y explicar cómo HMMER y Pfam lo usan para anotar dominios de proteínas.

En el ?? aprendimos a comparar *dos* secuencias. Pero la evolución rara vez nos entrega pares aislados: una familia de proteínas puede tener cientos o miles de miembros repartidos por todo el árbol de la vida, y lo que realmente queremos saber es qué tienen en común *todos* ellos. Una posición que se conserva en la hemoglobina humana, en la del ratón y en la mioglobina de un cachalote, separadas por cientos de millones de años de evolución, no se conserva por casualidad: probablemente sostiene el grupo hemo, estabiliza una hélice o participa en la función. El alineamiento de dos secuencias no puede distinguir ese residuo de uno que simplemente no ha tenido tiempo de cambiar; el alineamiento de muchas, sí.

Este capítulo recorre el camino que va de un conjunto de secuencias emparentadas a un *modelo estadístico* de la familia. Primero construiremos el alineamiento múltiple y veremos por qué la programación dinámica del ?? deja de ser practicable, lo que obliga a recurrir a heurísticas como el alineamiento progresivo (sección 4.1). Después resumiremos un conjunto de sitios alineados, sin huecos, en una matriz de peso posicional, y mediremos con la teoría de la información cuánta “señal” contiene cada posición (sección 4.2). Por último, generalizaremos esas matrices a modelos que admiten inserciones y delecciones, los modelos ocultos de Márkov de perfil, que son el motor de HMMER y de la base de datos Pfam (sección 4.3). Al final del capítulo, el HMM de pares que se anunció con el algoritmo de Gotoh (??) habrá encontrado su lugar natural.

4.1 Alineamiento múltiple: el método progresivo

 Colab · 4.1 · MSA: alineamiento progresivo

 repositorio del curso

La receta de la abuela

Cinco primos conservan, cada uno, una copia manuscrita de la receta de pan de la abuela. Con los años, cada copia ha acumulado cambios: alguien sustituyó la manteca por aceite, otro añadió una pizca de anís, otro olvidó copiar una línea entera. Si quisiera reconstruir la receta original, no empezaría

comparando las copias más distintas. Empezaría por las de las dos hermanas que viven juntas, casi idénticas, y fijaría entre ambas una versión común; luego la compararía con la del primo que vive en la misma ciudad; y solo al final se enfrentaría a la copia del pariente lejano. Cada decisión que tome en los primeros pasos (“esta línea se perdió aquí”) condicionará todas las siguientes. Ese procedimiento, con sus virtudes y su punto débil, es exactamente el alineamiento progresivo.

4.1.1 De dos secuencias a muchas

Un alineamiento múltiple extiende la idea del alineamiento por pares: se escriben k secuencias una debajo de otra, insertando huecos, de modo que los residuos que descienden de un mismo residuo ancestral queden en la misma columna.

Definición 4.1. (Alineamiento múltiple de secuencias)

Sean $x^{(1)}, \dots, x^{(k)}$ secuencias sobre un alfabeto $\mathcal{A}$. Un *alineamiento múltiple* es una matriz $\mathbf{M}$ de k filas y L columnas con entradas en $\mathcal{A} \cup \{-\}$ tal que (i) al eliminar los huecos de la fila p se recupera exactamente $x^{(p)}$ y (ii) ninguna columna está formada solo por huecos. Cada columna es una hipótesis de homología posicional.

Los MSA son la materia prima de buena parte de la bioinformática: de ellos se infieren árboles filogenéticos, se identifican residuos catalíticos, se predicen estructuras secundarias y terciarias, se construyen los perfiles y HMM de la sección 4.3 y se detectan sitios bajo selección. Por eso importa tanto hacerlos bien. Y, como en el caso de dos secuencias, “hacerlos bien” exige primero decidir cómo se puntúan.

4.1.2 La puntuación de suma de pares

La función objetivo más usada es la *suma de pares* (SP, *sum of pairs*): puntuar cada columna como la suma de las puntuaciones de todos los pares de residuos que contiene, con la misma matriz de sustitución que usamos para dos secuencias.

$$S_{\text{SP}}(\mathbf{M}) = \sum_{c=1}^L \sum_{p < r} w_p w_r \sigma(M_{pc}, M_{rc}), \quad \sigma(a, b) = \begin{cases} s(a, b) & a, b \in \mathcal{A} \\ -d & \text{uno de los dos es } - \\ 0 & a = b = - \end{cases} \quad (4.1)$$

DONDE	
M, M_{pc}	Alineamiento múltiple y su símbolo en la fila p , columna c .
L	Número de columnas del alineamiento.
$p < r$	Recorre los $\binom{k}{2}$ pares de filas distintas.
w_p	Peso de la secuencia p (todos iguales a 1 en la versión sin ponderar).
$s(a, b)$	Puntuación de sustitución (p. ej., BLOSUM62, ??).
d	Penalización por cada par residuo–hueco; en la práctica se usan huecos afines por par.

La SP es natural, pero tiene un defecto conocido: cuenta todos los pares por igual. Si un conjunto contiene diez variantes casi idénticas de una proteína de mamífero y una sola de levadura, la puntuación estará dominada por las comparaciones entre mamíferos, y el alineamiento se ajustará a ellos a costa de la secuencia divergente. Los pesos w_p corrigen ese sesgo. Una receta sencilla y eficaz es la de Henikoff y Henikoff (1994), que asigna más peso a las secuencias que aportan residuos poco comunes en cada columna:

$$w_p \propto \sum_{c=1}^L \frac{1}{r_c n_c(M_{pc})}, \tag{4.2}$$

DONDE	
r_c	Número de símbolos distintos presentes en la columna c .
$n_c(a)$	Número de veces que aparece el símbolo a en la columna c .

Una secuencia que comparte cada residuo con otras nueve recibe, en esa columna, una décima parte del crédito que recibe una que tiene un residuo único. Los pesos se normalizan después para que sumen uno.

Ejemplo 4.1. (Suma de pares de cuatro columnas de globinas)

Las cuatro primeras columnas del alineamiento de cinco globinas que construiremos en esta sección (figura 4.3) son

MYG_HUMAN	M	G	-	L
MYG_PHYMC	M	V	-	L
HBA_HUMAN	M	V	-	L
HBB_HUMAN	M	V	H	L
HBB1_MOUSE	M	V	H	L

Con BLOSUM62, $d = 8$ y pesos unitarios, hay $\binom{5}{2} = 10$ pares por columna:

- Columna 1: diez pares M-M, $10 \times 5 = 50$.
- Columna 2: cuatro pares G-V (-3) y seis V-V (+4): $-12 + 24 = 12$.
- Columna 3: tres pares hueco-hueco (o), seis pares hueco-H (-8) y un par H-H (+8): $-48 + 8 = -40$.
- Columna 4: diez pares L-L, $10 \times 4 = 40$.

El total es $50 + 12 - 40 + 40 = 62$. Los pesos de Henikoff de las cinco filas (tratando el hueco como un símbolo más) resultan 0,267, 0,173, 0,173, 0,194 y 0,194: la mioglobina humana recibe más peso porque su G es única en la columna 2.

4.1.3 Programación dinámica en k dimensiones

La recurrencia de Needleman-Wunsch (??) se generaliza sin dificultad conceptual. En lugar de una tabla bidimensional, llenamos un hipercubo con una celda por cada tupla de prefijos $(i_1, \dots, i_k)$. La última columna de un alineamiento óptimo de esos prefijos contiene, en cada fila, o bien el último residuo del prefijo, o bien un hueco; hay por tanto $2^k - 1$ posibilidades (se excluye la columna de solo huecos):

$$F(\mathbf{i}) = \max_{\substack{\delta \in \{0,1\}^k \\ \delta \neq \mathbf{0}}} \left\{ F(\mathbf{i} - \delta) + \text{SP}(\text{columna definida por } \delta) \right\}, \quad (4.3)$$

DONDE

$\mathbf{i} = (i_1, \dots, i_k)$	Vector de longitudes de los prefijos considerados.
δ	Vector binario: $\delta_p = 1$ si la fila p aporta su residuo $x_p^{(p)}$ a la columna; $\delta_p = 0$ si aporta un hueco.
$F(\mathbf{i})$	Mejor puntuación SP de un alineamiento de los k prefijos.

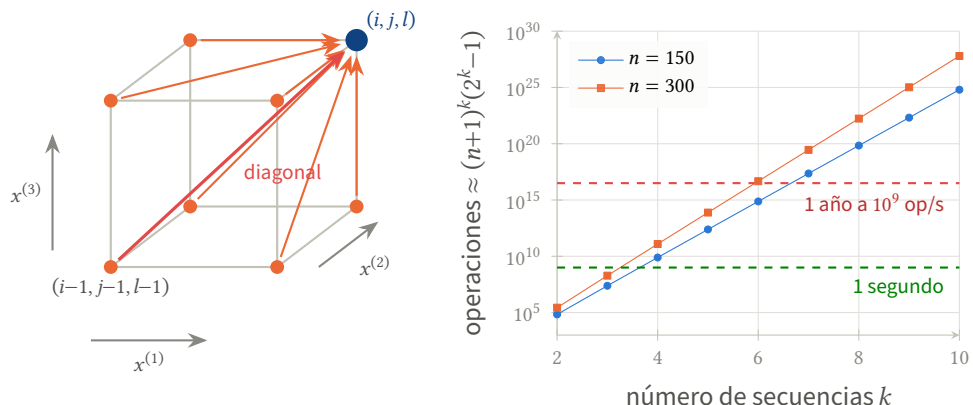


Figura 4.1. El costo del alineamiento múltiple exacto. **Izquierda:** con tres secuencias, cada celda (i, j, l) del cubo de programación dinámica depende de $2^3 - 1 = 7$ vecinas (puntos naranjas): la diagonal principal (rojo) coloca un residuo de cada secuencia en la columna; las otras seis combinan residuos y huecos. **Derecha:** el número de operaciones crece exponencialmente con k . Con proteínas de 150 residuos, la línea de un año de cómputo se cruza ya con siete secuencias; ninguna mejora de hardware alcanzará a una curva así.

Si todas las secuencias miden n , el hipercubo tiene $(n + 1)^k$ celdas, cada una consulta $2^k - 1$ vecinas y cada columna candidata requiere $\binom{k}{2}$ sumas. El coste total es

$$T(n, k) = O(k^2 2^k n^k) \text{ en tiempo,} \quad O(n^k) \text{ en memoria.} \quad (4.4)$$

Para las cinco globinas de este capítulo ($n \approx 150$) eso son unas $151^5 \approx 7,9 \times 10^{10}$ celdas, más de 300 gigabytes de memoria con enteros de 4 bytes, y $2,4 \times 10^{12}$ transiciones. Con diez secuencias, el número de celdas ($\approx 6 \times 10^{21}$) supera la memoria de todos los ordenadores del planeta juntos (figura 4.1).

¿Se trata solo de que no hemos encontrado un algoritmo más astuto? Probablemente no.

Teorema 4.1. (Intratabilidad del MSA)

Encontrar el alineamiento múltiple de puntuación SP óptima es un problema NP-difícil; su versión de decisión es NP-completa aun con matrices de puntuación sencillas (Wang & Jiang, 1994).

En consecuencia, salvo que $P = NP$, no existe un algoritmo que resuelva el MSA exacto en tiempo polinómico en k . Wang y Jiang (1994) demostraron además la

NP-completitud del *alineamiento sobre un árbol*, una variante que puntúa el MSA a lo largo de una filogenia en lugar de sumar todos los pares.

Podar el hipercubo: la cota de Carrillo-Lipman

No todo está perdido para valores pequeños de k . Carrillo y Lipman (1988) observaron que la proyección del MSA óptimo sobre cada par de secuencias es un alineamiento por pares cuya puntuación no puede quedar muy por debajo del óptimo de ese par: si lo hiciera, la suma total sería inferior a la de un MSA heurístico ya conocido. Esta desigualdad define una región alrededor de la diagonal del hipercubo fuera de la cual el camino óptimo no puede pasar, y permite alinear exactamente una decena de proteínas cortas. Más allá, incluso la región podada crece de forma exponencial.

4.1.4 La heurística progresiva

La salida práctica se debe a Feng y Doolittle (1987), que propusieron construir el MSA *de manera progresiva*, siguiendo el orden de parentesco de las secuencias. El procedimiento, que con variaciones usan casi todos los programas actuales, tiene tres etapas:

1. **Distancias.** Se alinean todos los pares de secuencias (o se estima su parecido con un método rápido) y se convierte cada similitud en una distancia d_{ij} , por ejemplo $d_{ij} = 1 - \text{identidad}$.
2. **Árbol guía.** Con la matriz de distancias se construye un árbol mediante un método de agrupamiento (UPGMA o *neighbor-joining*, que estudiaremos a fondo en el capítulo de filogenia).
3. **Alineamiento progresivo.** Se recorren los nodos internos del árbol desde las hojas hacia la raíz; en cada nodo se alinean los dos subalineamientos hijos mediante programación dinámica, y el resultado se congela y se pasa al nodo padre.

UPGMA, el método de agrupamiento más simple, une en cada paso los dos grupos A y B más cercanos y actualiza las distancias como promedio ponderado por tamaño:

$$d(A \cup B, C) = \frac{|A|d(A, C) + |B|d(B, C)}{|A| + |B|}, \quad h(A \cup B) = \frac{1}{2}d(A, B). \quad (4.5)$$

DONDE

A, B, C	Grupos (conjuntos de secuencias) en el paso actual.
$ A $	Número de secuencias del grupo A .
$h(\cdot)$	Altura del nodo nuevo en el dendrograma.

El árbol guía no pretende ser una filogenia correcta; solo es un *orden de trabajo* razonable: alinear primero lo fácil (lo más parecido) y dejar para el final lo difícil.

Alinear perfiles. El paso tres exige alinear no dos secuencias, sino dos *alineamientos*. Para ello cada subalineamiento se resume en un *perfil*: para cada columna u , el vector de frecuencias $f_u(a)$ de cada símbolo. La programación dinámica es idéntica a la de Needleman-Wunsch o Gotoh; solo cambia la puntuación de “alinear la columna u del perfil P con la columna v del perfil Q ”, que se toma como la SP media entre ambas:

$$S(P_u, Q_v) = \sum_{a \in \mathcal{A}} \sum_{b \in \mathcal{A}} f_u^P(a) f_v^Q(b) s(a, b). \quad (4.6)$$

DONDE

P_u, Q_v	Columna u del perfil P y columna v del perfil Q .
$f_u^P(a)$	Fracción (posiblemente ponderada) de secuencias de P con el símbolo a en la columna u .

Alinear una secuencia con otra es el caso particular en que ambos perfiles tienen una sola fila y f es una función indicadora. Como una columna de perfil es una distribución y no una letra, la ecuación (4.6) ya anticipa la idea central de la sección 4.3: puntuar *posiciones* con distribuciones propias.

Ejemplo 4.2. (El árbol guía de cinco globinas)

Las distancias de la figura 4.2 muestran dos parejas muy cercanas: las mioglobinas de humano y cachalote ($d = 0,156$) y las β -globinas de humano y ratón ($d = 0,197$). UPGMA procede así:

1. Une las mioglobinas, a altura $0,156/2 = 0,078$.
2. Une las β -globinas, a altura $0,197/2 \approx 0,099$.

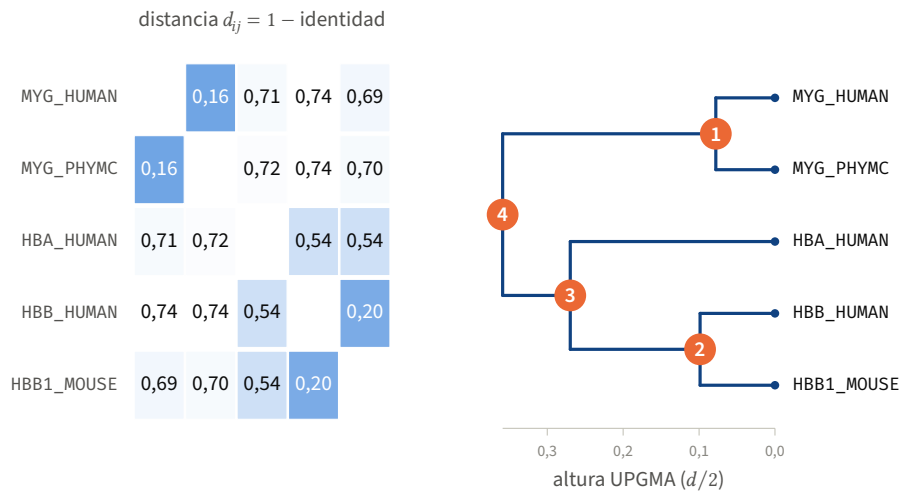


Figura 4.2. Las dos primeras etapas del alineamiento progresivo aplicadas a cinco globinas reales de UniProt. **Izquierda:** distancias $d_{ij} = 1 - \text{identidad}$ calculadas a partir de alineamientos globales por pares (BLOSUM62, huecos $-10/-1$); cuanto más oscuro, más parecidas. **Derecha:** árbol guía UPGMA; los círculos naranjas indican el *orden* en que se alinean los perfiles. Primero se unen las dos mioglobinas (1) y las dos cadenas β (2); luego la cadena α se incorpora a las β (3) y, por último, se alinean entre sí los dos grandes perfiles (4). Calculado con `figuras/cap04/generar.py`.

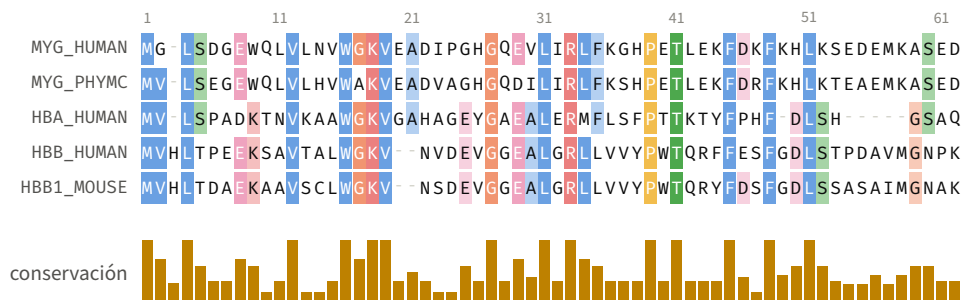


Figura 4.3. Primeras 62 columnas del alineamiento progresivo de las cinco globinas, calculado con perfiles y huecos afines. Los residuos se colorean por clase fisicoquímica (azul, hidrofóbicos; rojo, básicos; magenta, ácidos; verde, polares; naranja, glicina; amarillo, prolina; aqua, histidina y tirosina) cuando al menos el 60 % (claro) u 80 % (intenso) de la columna comparte el residuo. Las barras inferiores miden la conservación, $1 - H_c / \log_2 5$, penalizada por los huecos. Destacan el triptófano invariante de la columna 16 y el hueco de cinco posiciones de la cadena α (columnas 54–58), una delección real de la hemoglobina α respecto a la β .

3. Actualiza la distancia de HBA_HUMAN al grupo β con la ecuación (4.5): $\frac{1}{2}(0,536 + 0,543) = 0,539$. Es la menor, así que la cadena α se une a las β a altura 0,270.
4. Quedan dos grupos (mioglobinas y hemoglobinas); su distancia es la media de los seis pares cruzados, $4,305/6 \approx 0,718$, y la raíz queda a altura 0,359.

El orden de alineamiento reproduce la biología: las cadenas α y β de la hemoglobina proceden de una duplicación génica más reciente que la que separó a la mioglobina. El MSA resultante, de 155 columnas, tiene una puntuación SP de 2473 (BLOSUM62, huecos afines $-10/-1$ por par).

De Feng-Doolittle a la familia Clustal

El artículo de Feng y Doolittle (1987) tenía un objetivo filogenético: obtener alineamientos coherentes para construir árboles. Un año después, Higgins y Sharp (1988) publicaron CLUSTAL, un paquete diseñado para ejecutarse en los microordenadores personales de la época, que combinaba alineamientos rápidos por pares, un dendro-

grama y alineamiento progresivo de perfiles. Su sucesor, CLUSTAL W (Thompson et al., 1994), añadió tres ideas que siguen vigentes: ponderar las secuencias para compensar la sobrerrepresentación, penalizar los huecos de forma distinta según la posición (más baratos donde ya hay huecos o en regiones hidrofílicas) y elegir la matriz de sustitución según la divergencia de las secuencias que se alinean en cada nodo.

4.1.5 “Una vez un hueco, siempre un hueco”

El alineamiento progresivo es *codicioso*: cada nodo toma la mejor decisión local y la congela. Feng y Doolittle (1987) lo convirtieron en regla explícita: una vez que un hueco se introduce en un subalineamiento, no se elimina en pasos posteriores. La regla es sensata (los huecos entre secuencias cercanas son los más fiables) pero tiene un costo evidente: un error cometido al alinear las dos primeras secuencias se propaga a todo el alineamiento final, y ninguna información aportada después por las demás secuencias puede corregirlo.

Errores que se heredan

Si en el paso 1 de la figura 4.2 la programación dinámica hubiera colocado un hueco de las mioglobinas un par de posiciones a la izquierda de su lugar correcto (un empate de puntuación basta), ese hueco seguiría allí al final, aunque las tres hemoglobinas “votaran” por otra ubicación. Conviene revisar los huecos y comparar varios programas.

La historia de los programas posteriores es, en buena medida, la historia de los intentos de mitigar este problema sin perder la velocidad del método progresivo (figura 4.4).

Refinamiento iterativo. Una vez obtenido un MSA, se divide el árbol guía quitando una rama, lo que parte las secuencias en dos grupos; se extrae el subalineamiento de cada grupo, se eliminan las columnas que hayan quedado solo con huecos y se vuelven a alinear los dos perfiles. Si la SP mejora, se acepta el cambio. MUSCLE (Edgar, 2004) aplica este refinamiento restringido por el árbol tras dos etapas progresivas: la primera usa distancias rápidas por conteo de k -meros; la segunda recalcula el árbol a partir del primer MSA. Su función de puntuación perfil-perfil, la *log-expectation*, sustituye a la SP media de la ecuación (4.6).

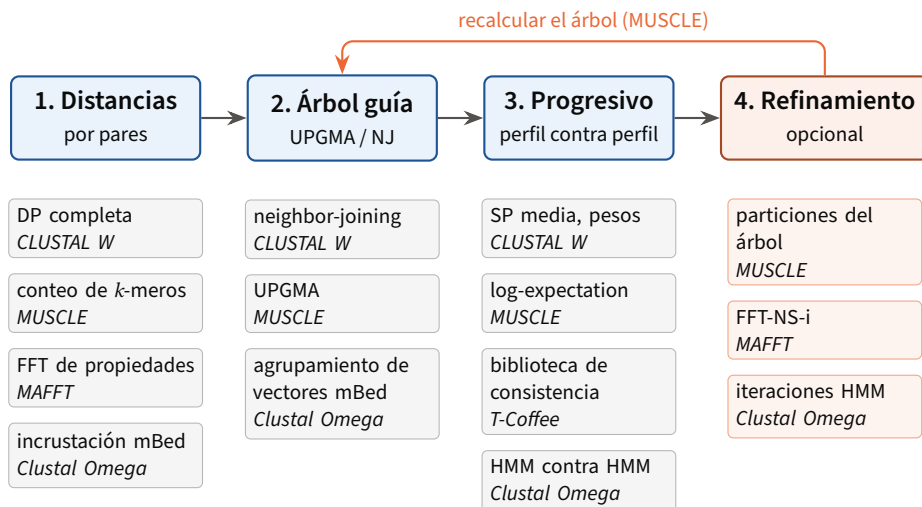


Figura 4.4. Casi todos los programas modernos de MSA comparten el esqueleto progresivo de Feng y Doolittle (1987) y se distinguen en cómo implementan cada etapa. Debajo de cada caja se indican, en cursiva, los programas que popularizaron cada variante. Las etapas 1 y 2 determinan la *velocidad*; las etapas 3 y 4, la *exactitud*. La flecha superior representa la segunda iteración de MUSCLE, que reconstruye el árbol a partir del primer alineamiento.

Consistencia. T-Coffee (Notredame et al., 2000) ataca el problema desde otro ángulo: antes de alinear nada, construye una *biblioteca* con todos los alineamientos por pares, globales y locales. El peso de alinear el residuo $x_i^{(1)}$ con $x_j^{(2)}$ aumenta si existe una tercera secuencia $x^{(3)}$ en la que ambos residuos se alinean con un mismo residuo $x_l^{(3)}$ (*extensión* de la biblioteca). Así, cuando el método progresivo alinea dos secuencias, “consulta” implícitamente a todas las demás, y la decisión codiciosa deja de serlo tanto.

Velocidad. MAFFT (Kato et al., 2002) convierte cada proteína en una señal numérica de volumen y polaridad de sus residuos y usa la transformada rápida de Fourier para localizar en segundos las regiones homólogas, que luego se alinean con programación dinámica restringida; ofrece un modo progresivo (FFT-NS-2) y uno con refinamiento iterativo (FFT-NS-i). Clustal Omega (Sievers et al., 2011) escala a conjuntos de decenas de miles de secuencias o más construyendo el árbol guía a partir de distancias a un subconjunto de secuencias de referencia (mBed) y alineando los perfiles como HMM contra HMM.

¿Cuál es el mejor? La pregunta solo tiene respuesta empírica. Los programas se comparan con *benchmarks* de alineamientos de referencia, casi siempre derivados de superposiciones de estructuras tridimensionales, como BALiBASE o PREFAB; la exactitud se mide como la fracción de pares de residuos (o de columnas) de la referencia que el programa reproduce. La revisión de Edgar y Batzoglou (2006) resume las lecciones de esas comparaciones: la consistencia y el refinamiento mejoran la exactitud sobre todo en familias divergentes (por debajo de un 25–30 % de identidad), donde ningún método es fiable, mientras que con secuencias cercanas todos los programas dan resultados muy parecidos.

```
msa_globinas.py

import subprocess
from Bio import AlignIO

# Clustal Omega: FASTA de entrada, formato Clustal de salida
subprocess.run(["clustalo", "-i", "globinas.fasta",
               "-o", "globinas.aln", "--outfmt=clu",
               "--force"], check=True)
aln = AlignIO.read("globinas.aln", "clustal")
print(aln.get_alignment_length(), "columnas")
for fila in aln:
    print(f"{fila.id:12s} {str(fila.seq[:60])}")

filas = [str(f.seq) for f in aln]
print("SP =", suma_de_pares(filas))          # ver ejercicio 4.2
print("pesos =", pesos_henikoff(filas))
```

Las funciones `suma_de_pares` y `pesos_henikoff` implementan la ecuación (4.1) con penalización lineal y la ecuación (4.2); en el notebook de la lección se propone comparar los MSA de Clustal Omega, MAFFT (`mafft --auto`) y MUSCLE sobre las mismas globinas y visualizar las columnas en las que discrepan.

4.2 Motivos, matrices de peso posicional y logos



Reconocer una firma

Nadie firma dos veces exactamente igual. Aun así, un empleado de banco experimentado reconoce la firma de un cliente: sabe que la inicial siempre tiene el mismo bucle, que el trazo final a veces sube y a veces no, y que el punto sobre la *i* es opcional. En su cabeza no hay una firma “correcta”, sino una idea de qué rasgos son obligatorios, cuáles son frecuentes y cuáles dan igual. Una proteína que se une al ADN hace algo muy parecido: tolera variación en algunas posiciones de su sitio de unión y es muy exigente en otras. Esta sección enseña a escribir esa “idea de la firma” como una tabla de números.

4.2.1 Motivos y secuencias consenso

Un *motivo* es un patrón corto y recurrente, de ADN o de proteína, asociado a una función: el sitio donde se une un factor de transcripción, una señal de corte y empalme, el sitio de unión al ribosoma o un segmento catalítico. A diferencia de los dominios completos de la sección 4.3, los motivos de ADN suelen medir entre 6 y 20 posiciones, no contienen huecos y se presentan como *sitios alineados* de igual longitud.

Tomemos un ejemplo clásico. La proteína de unión a la caja TATA (TBP) reconoce, unas 30 bases antes del inicio de la transcripción de muchos genes eucariotas, una secuencia rica en **A** y **T**. La base de datos JASPAR (Castro-Mondragon et al., 2022) guarda su perfil MA0108.2, construido a partir de 389 sitios alineados de 15 bases. La forma más antigua de resumir esos sitios es la *secuencia consenso*, que toma la base mayoritaria de cada columna:

GTATAAAA**GG**GGGG

El consenso es cómodo, pero pierde casi toda la información. No distingue una columna en la que el 96 % de los sitios tiene una **T** (la posición 4) de otra en la que la **G** gana con apenas un 39 % (la posición 1). Y un sitio real con una sola discrepancia respecto al consenso puede ser mucho mejor, o mucho peor, que otro con tres. Necesitamos una representación *cuantitativa*.

4.2.2 De sitios a frecuencias: conteos y pseudoconteos

Sea un conjunto de N sitios alineados de longitud W . La *matriz de conteos* (o de frecuencias de posición, PFM) registra cuántas veces aparece cada base en cada columna:

$$n_{b,i} = \sum_{s=1}^N \mathbb{1}[x_i^{(s)} = b], \quad b \in \{A, C, G, T\}, \quad i = 1, \dots, W. \quad (4.7)$$

DONDE

$x_i^{(s)}$ Base en la posición i del sitio s .

$n_{b,i}$ Número de sitios con la base b en la posición i ; $\sum_b n_{b,i} = N$.

La estimación de máxima verosimilitud de la probabilidad de la base b en la posición i es $n_{b,i}/N$. Con pocas observaciones, esta estimación es peligrosa: si una base no aparece en ninguno de los sitios conocidos, su probabilidad estimada es exactamente cero, y cualquier sitio futuro que la contenga será declarado *imposible*. La solución estándar es sumar *pseudoconteos*:

$$f_{b,i} = \frac{n_{b,i} + \alpha_b}{N + \sum_{b'} \alpha_{b'}}. \quad (4.8)$$

DONDE

$f_{b,i}$ Probabilidad estimada de la base b en la posición i .

α_b Pseudoconteo de la base b (p. ej., $\alpha_b = 0,5$, o $\alpha_b = \beta q_b$ proporcional al fondo).

Desde el punto de vista bayesiano, la ecuación (4.8) es la media posterior de una distribución multinomial con una distribución previa de Dirichlet de parámetros α_b : los pseudoconteos son “observaciones imaginarias” que expresan lo que creíamos antes de ver los datos. Cuando N es grande pesan poco; cuando N es pequeño, dominan. Para proteínas, en las que el alfabeto tiene 20 letras y los datos son siempre escasos, conviene usar mezclas de distribuciones de Dirichlet que codifican preferencias fisicoquímicas (Sjölander et al., 1996): si una columna contiene solo I y L, la previa “sabe” que una V es mucho más plausible que una D.

4.2.3 La matriz de peso posicional

La pregunta que queremos responder ante una ventana $y = y_1 \dots y_W$ de una secuencia es la misma del ??: ¿es más verosímil bajo el modelo del motivo o bajo el

modelo de fondo? Si suponemos que las posiciones son independientes, la razón de verosimilitudes se factoriza y su logaritmo es una suma:

$$W_{b,i} = \log_2 \frac{f_{b,i}}{q_b}, \quad S(y) = \log_2 \frac{\mathbb{P}(y \mid \text{motivo})}{\mathbb{P}(y \mid \text{fondo})} = \sum_{i=1}^W W_{y_i,i}. \quad (4.9)$$

DONDE	
$W_{b,i}$	Entrada de la matriz de peso posicional (PWM o PSSM), en bits.
q_b	Frecuencia de fondo de la base b (p. ej., 0,25, o la composición del genoma).
y	Ventana candidata de longitud W .
$S(y)$	Puntuación de la ventana: log-razón de verosimilitudes motivo/fondo.

Esta es exactamente la lógica de las matrices de sustitución (??), aplicada ahora a una sola secuencia contra un modelo *posicional*. Una entrada positiva indica que la base es más frecuente en los sitios que en el fondo; una negativa, lo contrario; y la suma de las entradas a lo largo de la ventana decide.

 **Un perceptrón para los ribosomas**

Las matrices de peso nacieron antes que su interpretación probabilística. Stormo et al. (1982) buscaban distinguir los verdaderos sitios de inicio de la traducción en *Escherichia coli* de los codones AUG que no se usan, y entrenaron para ello un *perceptrón*: una tabla de pesos, uno por base y posición, que suma sus valores sobre la ventana y compara el total con un umbral. Los pesos aprendidos resultaron ser un buen clasificador y, a la vez, una descripción legible del sitio. Dos décadas después, Stormo (2000) mostró que, bajo el supuesto de aditividad, la puntuación de una PWM es proporcional a la energía libre de unión de la proteína con el sitio, lo que da a la matriz un significado físico además del estadístico.

Ejemplo 4.3. (La columna 3 del perfil de TBP)

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	-0,7	-2,6	1,9	-4,8	1,9	1,5	1,9	1,2	0,7	-0,8	-0,2	-0,2	-0,2	-0,5	-0,3
C	0,6	-1,1	-7,6	-3,2	-7,6	-7,6	-4,8	-5,3	-1,1	0,5	0,6	0,4	0,3	0,1	0,1
G	0,6	-2,4	-5,3	-5,3	-4,2	-7,6	-2,3	-1,1	0,7	0,6	0,4	0,4	0,4	0,5	0,5
T	-1,6	1,7	-1,5	1,9	-1,7	0,3	-3,9	0,3	-1,5	-1,0	-1,6	-0,9	-0,7	-0,4	-0,5
R_i (bits)	0,23	0,97	1,49	1,69	1,49	1,08	1,51	0,61	0,28	0,16	0,17	0,08	0,06	0,06	0,05

Figura 4.5. Matriz de peso posicional $W_{b,i} = \log_2(f_{b,i}/0,25)$ del perfil de TBP (JASPAR MA0108.2, 389 sitios), con pseudoconteos $\alpha_b = 0,5$. Azul: bases favorecidas; rojo: desfavorecidas. El núcleo **TATAAA** (posiciones 2–7) concentra los pesos extremos; en la posición 3, la **C** vale $-7,6$ bits: verla en un sitio multiplica su verosimilitud por $2^{-7,6} \approx 1/190$. La fila inferior da el contenido de información R_i de cada columna (ecuación (4.11)); las posiciones 10–15 apenas aportan señal.

En la posición 3 del perfil MA0108.2 los conteos son $n_A = 352$, $n_C = 0$, $n_G = 2$, $n_T = 35$ ($N = 389$). Con $\alpha_b = 0,5$ el denominador de la ecuación (4.8) es $389 + 2 = 391$ y

$$\begin{aligned} f_{A,3} &= \frac{352,5}{391} = 0,9015, & f_{C,3} &= \frac{0,5}{391} = 0,0013, \\ f_{G,3} &= \frac{2,5}{391} = 0,0064, & f_{T,3} &= \frac{35,5}{391} = 0,0908. \end{aligned}$$

Con fondo uniforme, los pesos son $W_{A,3} = \log_2(0,9015/0,25) = 1,85$, $W_{C,3} = -7,61$, $W_{G,3} = -5,29$ y $W_{T,3} = -1,46$ bits. Sin pseudoconteos, $W_{C,3}$ sería $-\infty$ y un único error de secuenciación en esa posición anularía cualquier sitio. Sumando el máximo de cada columna se obtiene la puntuación del consenso, 16,21 bits; la peor secuencia posible puntúa $-49,13$.

4.2.4 Entropía, información y logos

¿Cuánta “señal” hay en cada columna? Claude Shannon definió la incertidumbre de una variable aleatoria discreta como su *entropía* (Shannon, 1948). Aplicada a la columna i de un motivo,

$$H_i = - \sum_b f_{b,i} \log_2 f_{b,i} \quad (\text{bits}). \quad (4.10)$$

DONDE

H_i Entropía de la columna i : número medio de preguntas de sí o no necesarias para adivinar la base.

Una columna uniforme ($f_{b,i} = 1/4$) tiene la entropía máxima, $\log_2 4 = 2$ bits; una columna invariante, entropía cero. El *contenido de información* de la columna es la reducción de incertidumbre que produce conocer que estamos en un sitio:

$$R_i = \log_2 4 - (H_i + e_N). \quad (4.11)$$

DONDE

R_i Contenido de información de la posición i , entre 0 y 2 bits para ADN.

e_N Corrección por muestra pequeña: con pocos sitios, H_i se subestima sistemáticamente; $e_N \rightarrow 0$ cuando $N \rightarrow \infty$.

Schneider y Stephens (1990) usaron R_i para inventar el *sequence logo*: en cada posición se apilan las cuatro letras, ordenadas de menor a mayor frecuencia, con altura

$$h_{b,i} = f_{b,i} R_i, \quad (4.12)$$

de modo que la altura total de la pila es R_i y la proporción de cada letra dentro de la pila es su frecuencia. De un vistazo, el logo muestra *qué* posiciones importan (pilas altas) y *qué* base se prefiere en ellas (la letra de arriba). WebLogo (Crooks et al., 2004) popularizó la herramienta en la web y es hoy el estándar de facto.

Con un fondo no uniforme (un genoma rico en A/T, por ejemplo), la medida adecuada es la *entropía relativa* o divergencia de Kullback-Leibler entre la columna y el fondo:

$$D_i = D_{\text{KL}}(f_{\cdot,i} \| q) = \sum_b f_{b,i} \log_2 \frac{f_{b,i}}{q_b} \geq 0, \quad (4.13)$$

que se reduce a R_i (sin corrección) cuando $q_b = 1/4$. Esta cantidad tiene una interpretación que conecta las dos mitades de la sección.

Teorema 4.2. (Puntuación esperada de una PWM)

Si los sitios se generan con las frecuencias $f_{b,i}$ del propio modelo, la puntuación esperada de la ecuación (4.9) es $\mathbb{E}_f[S] = \sum_i D_{\text{KL}}(f_{\cdot,i} \| q)$. Si las ventanas proceden del fondo, $\mathbb{E}_q[S] = - \sum_i D_{\text{KL}}(q \| f_{\cdot,i}) \leq 0$.

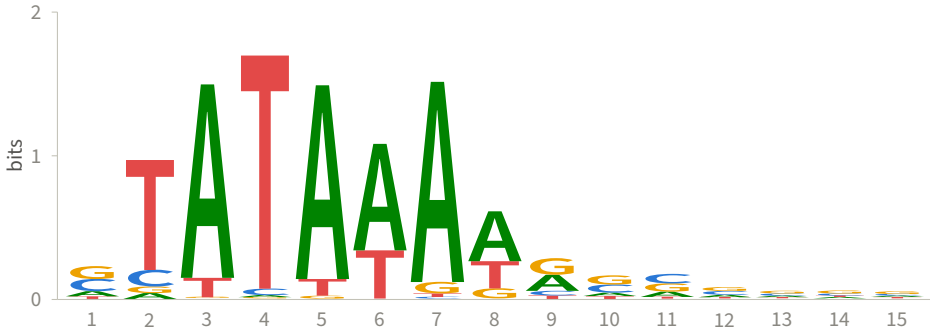


Figura 4.6. *Sequence logo* del perfil de TBP, dibujado en TikZ directamente a partir de las frecuencias de la figura 4.5: la altura de cada letra es $f_{b,i}R_i$ (ecuación (4.12)). Las posiciones 3–7 alternan A y T con más de 1,4 bits cada una; la posición 6 muestra que TBP tolera tanto A como T; la cola G/C de las posiciones 9–15 es casi ruido (menos de 0,3 bits). El contenido total del motivo es $\sum_i R_i \approx 9,9$ bits.

La demostración es una línea: por linealidad de la esperanza, $E_f[S] = \sum_i \sum_b f_{b,i} W_{b,i}$, que es la ecuación (4.13) sumada sobre las posiciones. El teorema dice que el contenido de información de un motivo es la *separación media*, en bits, entre sitios y fondo. Para TBP, $\sum_i D_i = 9,93$ bits, y una simulación de 20 000 sitios da una media de 9,95; las ventanas aleatorias promedian $-18,09$ bits, en perfecto acuerdo con $-\sum_i D_{\text{KL}}(q \| f_{\cdot,i}) = -18,09$ (figura 4.8). Un motivo con R bits aparece por azar, aproximadamente, una vez cada 2^R posiciones: el de TBP, una vez cada mil bases.

4.2.5 Buscar sitios en una secuencia

Con la PWM en la mano, buscar sitios es sencillo: se desliza una ventana de longitud W por la secuencia (y por su reverso complementario), se calcula $S(y)$ en cada posición y se reportan las que superan un umbral τ . El coste es $O(nW)$ para una secuencia de longitud n . La dificultad no es computacional, sino estadística: elegir τ .

La figura 4.8 muestra el dilema. Un umbral de 6 bits deja pasar solo un 0,19 % de las ventanas aleatorias, lo que parece poco; pero el genoma humano tiene unos $3,1 \times 10^9$ pares de bases y dos hebras, así que esperaríamos $2 \times 3,1 \times 10^9 \times 0,0019 \approx 1,2 \times 10^7$ “sitios” de TBP, la inmensa mayoría sin ninguna función. Esta abundancia de aciertos espurios no es un defecto de la PWM: refleja que la especificidad de un factor de transcripción *in vivo* depende también de la accesibilidad de la cromatina, de otros factores cercanos y de la posición respecto a los genes, información que la matriz no contiene.

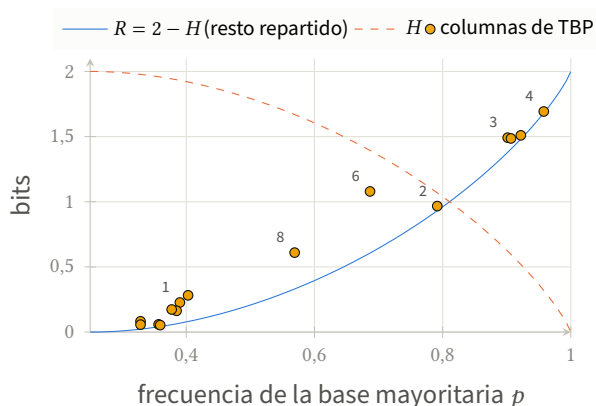


Figura 4.7. Entropía H (naranja) y contenido de información $R = 2 - H$ (azul) de una columna de ADN en la que la base mayoritaria tiene frecuencia p y las otras tres se reparten el resto por igual. Los puntos son las 15 columnas reales de TBP (se numeran algunas; las posiciones 9–15 se agrupan cerca de $p \approx 0,4$ y las 5 y 7 junto a la 3). Caen sobre la curva o por encima de ella, porque en ellas el resto no se reparte por igual (p. ej., la posición 6 mezcla **A** y **T**), y un reparto desigual *incrementa* la información. La relación no es lineal: pasar del 90 % al 99 % de conservación añade más información que pasar del 40 % al 60 %.

```

motivo_tbp.py

from Bio import motifs
from Bio.Seq import Seq

with open("MA0108.2.jaspar") as fh:           # descargado de JASPAR
    m = motifs.read(fh, "jaspar")
    pwm = m.counts.normalize(pseudocounts=0.5) # f con pseudoconteos
    pssm = pwm.log_odds()                     # W = log2(f / q)
    print(m.name, m.consensus)                # TBP GTATAAAAGGCGGGG
    print(f"máx={pssm.max:.2f} mín={pssm.min:.2f} "
          f"media en sitios={pssm.mean():.2f}") # 16.21 -49.13 9.93

prom = Seq("GGCCGCTATAAAAGGCGCGGCATGCGCCGCCATG")
for pos, score in pssm.search(prom, threshold=6.0):
    hebra = "+" if pos >= 0 else "-"          # pos < 0: hebra reversa
    print(pos, hebra, f"{score:.2f}")        # 5 + 15.55

```

El módulo `Bio.motifs` de Biopython (Cock et al., 2009) implementa las ecuaciones (4.8) y (4.9); su método `mean()` devuelve precisamente la puntuación esperada

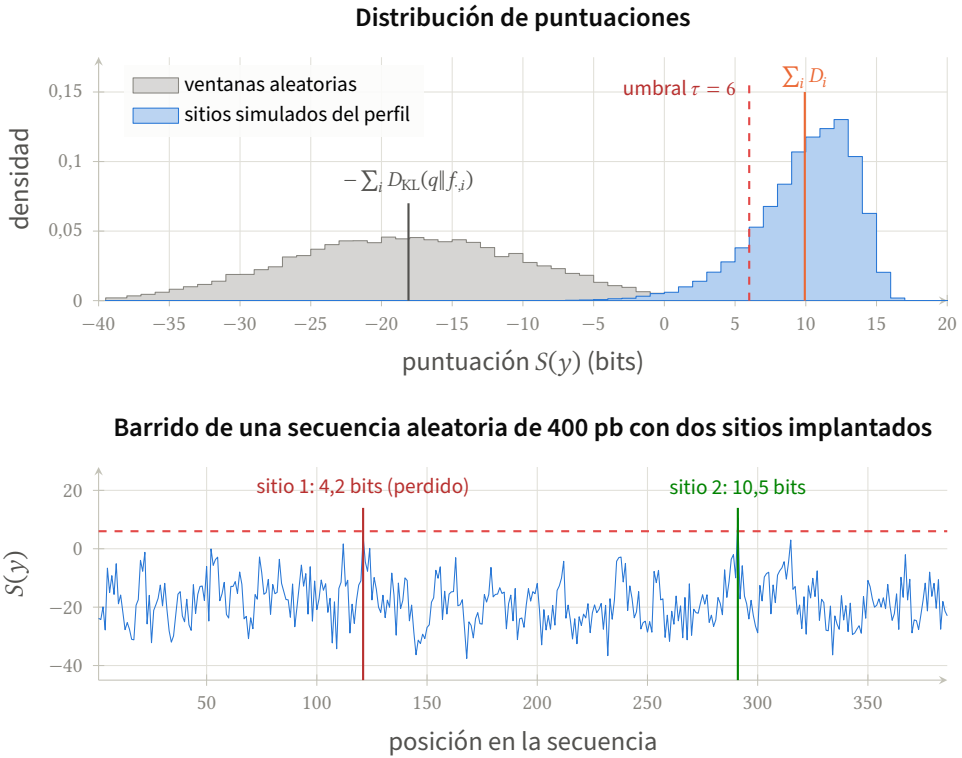


Figura 4.8. Arriba: distribución de $S(y)$ para 20 000 sitios generados a partir de las frecuencias de TBP (azul) y 20 000 ventanas de fondo uniforme (gris). Sus medias coinciden con las predicciones del teorema 4.2. Con $\tau = 6$ bits se recupera el 87 % de los sitios y solo pasa el 0,19 % de las ventanas aleatorias. Abajo: barrido de una secuencia aleatoria en la que se implantaron dos sitios muestreados del perfil. El sitio 2 (GTATAAGTAGCGAGG) destaca claramente; el sitio 1 (CCATAATAACTAGCG), también muestreado del modelo, puntúa solo 4,2 bits y cae bajo el umbral: un falso negativo tan legítimo como inevitable.

del teorema 4.2. La secuencia prom es un fragmento sintético con una caja TATA en la posición 5 (contando desde cero).

El supuesto de independencia

La ecuación (4.9) supone que cada posición contribuye de forma independiente a la unión. Es una aproximación sorprendentemente buena para muchos factores, pero falla cuando dos bases vecinas interactúan (por ejemplo, por la flexibilidad del ADN) o cuando el factor se une como dímero con espaciadores variables. En esos casos se usan modelos con dependencias entre posiciones, o los modelos de Márkov de la sección siguiente.

Descubrir motivos sin conocerlos

Hasta ahora partimos de sitios ya alineados. El problema inverso, encontrar un motivo desconocido compartido por un conjunto de secuencias (por ejemplo, promotores de genes coexpresados), es mucho más difícil, porque hay que estimar a la vez la PWM y la posición del sitio en cada secuencia. Lawrence et al. (1993) lo resolvieron con un muestreador de Gibbs: se elige al azar una posición en cada secuencia, se construye la PWM con todas menos una, se remuestra la posición de la excluida en proporción a $2^{S(y)}$ y se repite. El método converge hacia los motivos de alto contenido de información. Las alternativas deterministas se basan en el algoritmo EM (Dempster et al., 1977), el mismo principio que usaremos para entrenar HMM.

4.3 Modelos ocultos de Márkov y perfiles (HMMER)

 Colab · 4.3 · HMM y perfiles con HMMER

 repositorio del curso

Un partido por la radio

Imagine que escucha un partido de fútbol por una radio extranjera, en un idioma que no entiende. No ve el campo, pero oye el rugido del público: murmullo, silbidos, gritos. Cuando el equipo local ataca, los gritos son frecuentes; cuando defiende, predominan los silbidos. Además, la posesión no cambia a cada segundo: quien ataca suele seguir atacando un rato. Con esas dos regularidades (qué sonidos produce cada situación y cuánto duran

las situaciones) usted puede reconstruir, con bastante acierto, la historia del partido que no vio. Un modelo oculto de Márkov formaliza exactamente ese razonamiento: estados que no se observan, que persisten y cambian con ciertas probabilidades, y que emiten señales que sí se observan.

4.3.1 Estados, transiciones y emisiones

En bioinformática los “sonidos” son nucleótidos o aminoácidos, y los estados ocultos son categorías biológicas: región codificante o intergénica, isla CpG o ADN ordinario, hélice o lámina, posición 17 de un dominio o inserción. Eddy (2004) ofrece una introducción breve y muy clara; el tutorial de Rabiner (1989), escrito para el reconocimiento del habla, sigue siendo la referencia clásica.

Definición 4.2. (Modelo oculto de Márkov)

Un HMM con K estados sobre un alfabeto $\mathcal{A}$ es una terna $\theta = (\pi, \mathbf{A}, \mathbf{E})$ que genera un camino de estados $\pi_1 \pi_2 \dots \pi_L$ y una secuencia $x_1 \dots x_L$: el primer estado se elige con probabilidad π_k ; cada estado siguiente depende solo del anterior, con probabilidad a_{kl} ; y cada estado π_i emite el símbolo x_i con probabilidad $e_{\pi_i}(x_i)$. Se observa x ; el camino permanece oculto.

La probabilidad conjunta de una secuencia y un camino es el producto de las decisiones que los generaron:

$$P(x, \pi \mid \theta) = \pi_{\pi_1} e_{\pi_1}(x_1) \prod_{i=2}^L a_{\pi_{i-1}\pi_i} e_{\pi_i}(x_i). \quad (4.14)$$

DONDE

$\pi = \pi_1 \dots \pi_L$	Camino de estados ocultos (no confundir con π_k , la probabilidad inicial).
π_k	Probabilidad de empezar en el estado k .
a_{kl}	Probabilidad de transición del estado k al l ; $\sum_l a_{kl} = 1$.
$e_k(b)$	Probabilidad de que el estado k emita el símbolo b ; $\sum_b e_k(b) = 1$.
L	Longitud de la secuencia observada.

Usaremos a lo largo de la sección un modelo de juguete con dos estados para ADN: H, regiones ricas en G+C (como las islas CpG), y L, regiones pobres en G+C

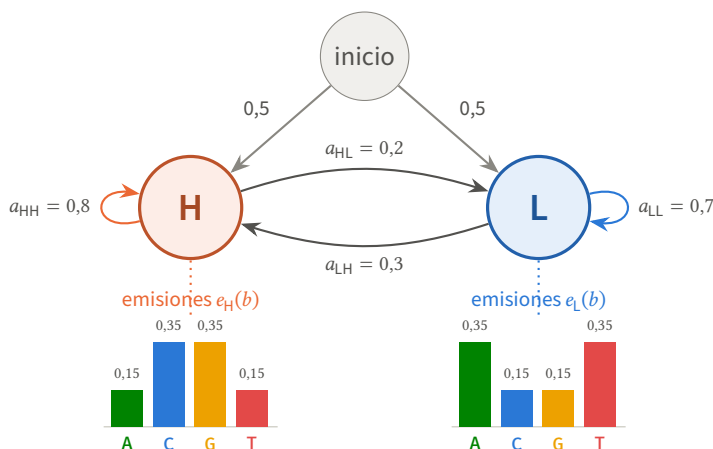


Figura 4.9. El modelo de juguete de esta sección. Dos estados ocultos, H (rico en G+C) y L (pobre), con transiciones que favorecen la permanencia en el mismo estado, y distribuciones de emisión opuestas (barras). Ninguna base individual revela el estado: una G es $0,35/0,15 \approx 2,3$ veces más probable en H que en L, pero puede proceder de ambos. La evidencia sobre el estado se acumula a lo largo de las posiciones consecutivas, porque los estados persisten.

(figura 4.9). El modelo plantea los tres problemas canónicos que Rabiner (1989) ordenó:

1. **Evaluación:** ¿cuál es la probabilidad $\mathbb{P}(x \mid \theta)$ de la secuencia, sumando sobre todos los caminos? (algoritmo *forward*).
2. **Decodificación:** ¿cuál es el camino de estados más probable? (algoritmo de Viterbi).
3. **Aprendizaje:** ¿qué parámetros θ hacen más probables las secuencias observadas? (algoritmo de Baum-Welch).

4.3.2 El algoritmo forward: sumar sobre todos los caminos

Hay K^L caminos posibles; para $K = 2$ y $L = 300$, unos 10^{90} . Por suerte, la estructura de Márkov permite reutilizar cálculos, igual que en la programación dinámica del ??, pero *sumando* en lugar de maximizar. Definimos la variable *forward*

como la probabilidad de haber emitido los primeros i símbolos y estar en el estado k :

$$f_k(1) = \pi_k e_k(x_1), \quad (4.15)$$

$$f_k(i+1) = e_k(x_{i+1}) \sum_{k=1}^K f_k(i) a_{kl}, \quad i = 1, \dots, L-1, \quad (4.16)$$

$$P(x | \theta) = \sum_{k=1}^K f_k(L). \quad (4.17)$$

DONDE

$f_k(i)$ $P(x_1 \dots x_i, \pi_i = k)$: probabilidad del prefijo terminando en el estado k .

K Número de estados.

El costo es $O(LK^2)$: en cada posición, cada estado suma las contribuciones de todos los estados anteriores. El algoritmo simétrico, *backward*, recorre la secuencia desde el final y calcula la probabilidad del sufijo dado el estado:

$$b_k(L) = 1, \quad b_k(i) = \sum_{l=1}^K a_{kl} e_l(x_{i+1}) b_l(i+1), \quad i = L-1, \dots, 1. \quad (4.18)$$

DONDE

$b_k(i)$ $P(x_{i+1} \dots x_L | \pi_i = k)$: probabilidad del sufijo dado que en i estamos en k .

Combinando ambos se obtiene la *probabilidad posterior* de cada estado en cada posición, la cantidad más útil en la práctica:

$$P(\pi_i = k | x, \theta) = \frac{f_k(i) b_k(i)}{P(x | \theta)}. \quad (4.19)$$

Ejemplo 4.4. (Forward a mano)

Con el modelo de la figura 4.9 y $x = \text{GCGATTAT}$:

$$f_H(1) = 0,5 \times 0,35 = 0,175, \quad f_L(1) = 0,5 \times 0,15 = 0,075.$$

Para la segunda base (C), según la ecuación (4.16),

$$f_H(2) = 0,35 (0,175 \times 0,8 + 0,075 \times 0,3) = 0,35 \times 0,1625 = 0,0569,$$

$$f_L(2) = 0,15 (0,175 \times 0,2 + 0,075 \times 0,7) = 0,15 \times 0,0875 = 0,0131.$$

Continuando hasta $i = 9$ y sumando (ecuación (4.17)) se obtiene $P(x | \theta) = 5,10 \times 10^{-6}$, es decir, $\log_2 P(x) = -17,58$ bits. Las probabilidades posteriores del estado H (ecuación (4.19)) son 0,80, 0,87, 0,86, 0,75 en las cuatro primeras posiciones (GCGC) y caen a 0,36, 0,22, 0,18, 0,18, 0,24 en el tramo ATTAT.

⚠ Desbordamiento por abajo

Las probabilidades forward decrecen exponencialmente con L . Para una secuencia de 5000 bases como la que usaremos para entrenar el modelo, $P(x) \approx 2^{-9712}$, un número que ningún formato de coma flotante puede representar (el menor double positivo es del orden de 2^{-1074}). Hay dos soluciones: trabajar en escala logarítmica, sustituyendo las sumas por $\log \sum \exp$ (p. ej., `numpy.logaddexp`), o *reescalar* $f_k(i)$ en cada posición dividiendo por $c_i = \sum_k f_k(i)$ y acumulando $\log P(x) = \sum_i \log c_i$. HMMER usa una variante de la segunda, el *sparse rescaling* (Eddy, 2011).

4.3.3 El algoritmo de Viterbi: el mejor camino

Si en lugar de sumar sobre caminos queremos el *más probable*, basta con cambiar la suma de la ecuación (4.16) por un máximo y recordar de dónde vino cada máximo. El algoritmo fue propuesto por Viterbi (1967) para decodificar códigos convolucionales en telecomunicaciones.

Teorema 4.3. (Algoritmo de Viterbi)

Sea $v_k(i)$ la probabilidad del camino más probable que emite $x_1 \cdots x_i$ y termina en el estado k . Entonces

$$v_k(1) = \pi_k e_k(x_1), \quad (4.20)$$

$$v_l(i+1) = e_l(x_{i+1}) \max_k [v_k(i) a_{kl}], \quad (4.21)$$

y $P(x, \pi^*) = \max_k v_k(L)$, donde π^* es el camino óptimo. El costo es $O(LK^2)$ en tiempo y $O(LK)$ en memoria.

DONDE

$v_k(i)$	Probabilidad del mejor camino parcial que termina en k en la posición i .
π^*	Camino de Viterbi: $\arg \max_{\pi} P(x, \pi)$.
$\text{ptr}_i(l)$	Puntero: el estado k que maximizó la ecuación (4.21) para llegar a l en i .

El camino se recupera, como en Needleman-Wunsch, con un *traceback*: $\pi_L^* = \arg \max_k v_k(L)$ y, hacia atrás, $\pi_{i-1}^* = \text{ptr}_i(\pi_i^*)$. En la práctica se trabaja con logaritmos, de modo que los productos se vuelven sumas y la ecuación (4.21) adopta la forma exacta de una recurrencia de programación dinámica: $V_l(i+1) = \log e_l(x_{i+1}) + \max_k [V_k(i) + \log a_{kl}]$.

Ejemplo 4.5. (Viterbi frente a posterior)

En la figura 4.10, $\log_2 v_L(9) = -19,98$ supera a $\log_2 v_H(9) = -22,42$, así que el camino termina en L. Su probabilidad conjunta es $2^{-19,98}$, y como $P(x) = 2^{-17,58}$, la probabilidad posterior del camino completo es

$$P(\pi^* | x) = \frac{2^{-19,98}}{2^{-17,58}} = 2^{-2,40} \approx 0,19.$$

El camino más probable acapara menos de una quinta parte de la probabilidad: el 81 % restante se reparte entre los otros 511 caminos. Por eso, cuando interesa la etiqueta de cada posición más que un camino global coherente, se prefiere la *decodificación posterior*, que asigna a cada i el estado con mayor ecuación (4.19).

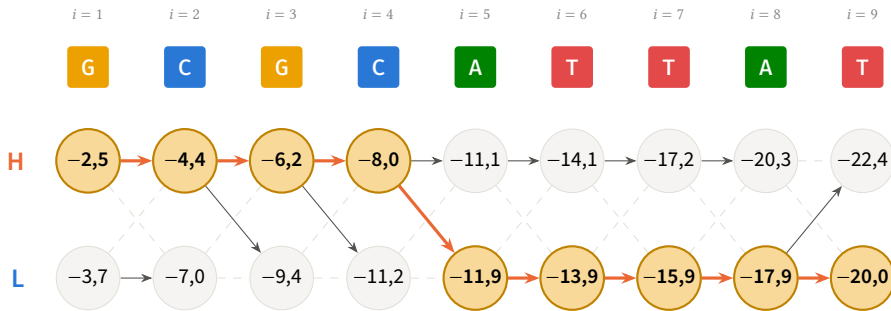


Figura 4.10. Enrejado (*trellis*) de Viterbi para $x = \text{GCGCATTAT}$ con el modelo de la figura 4.9. Cada nodo muestra $\log_2 v_k(i)$; las flechas grises indican el predecesor elegido por el máximo de la ecuación (4.21) y las discontinuas, las alternativas descartadas. El camino óptimo (naranja, nodos amarillos) es HHHHLLLLL: la región GCGC se asigna a H y el tramo ATTAT a L. Obsérvese que en $i = 5$ el mejor predecesor de L ya es H: el cambio de estado “se paga” una sola vez, con $\log_2 0,2$.

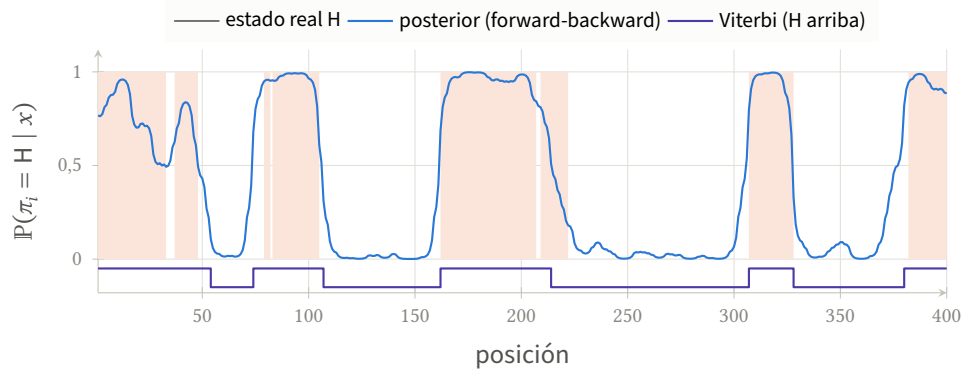


Figura 4.11. Decodificación de 400 bases simuladas con el modelo de la figura 4.9, pero con transiciones más persistentes ($a_{HH} = a_{LL} = 0,98$). Fondo naranja: tramos generados realmente por H. Curva azul: probabilidad posterior de H; su umbral en 0,5 acierta el estado en el 94 % de las posiciones. Escalón violeta (abajo): camino de Viterbi, con un 92,5 % de acierto. Ambos métodos detectan los bloques largos; los tramos cortos y los bordes son inciertos, y solo la posterior lo dice.

```

forward_viterbi.py

import numpy as np

def forward_viterbi(x, A, E, pi, alfabeto="ACGT"):
    """Log-verosimilitud (forward) y camino de Viterbi, en log2."""
    o = [alfabeto.index(c) for c in x]
    lA, lE = np.log2(A), np.log2(E)
    f = np.log2(pi) + lE[:, o[0]] # inicio forward (log)
    v, ptr = f.copy(), []
    for s in o[1:]:
        f = lE[:, s] + np.logaddexp2.reduce(f[:, None] + lA, axis=0)
        cand = v[:, None] + lA # K x K
        ptr.append(cand.argmax(axis=0))
        v = lE[:, s] + cand.max(axis=0)
    camino = [int(v.argmax())]
    for p in reversed(ptr): # traceback
        camino.append(int(p[camino[-1]]))
    return np.logaddexp2.reduce(f), v.max(), camino[::-1]

A = np.array([[0.8, 0.2], [0.3, 0.7]])
E = np.array([[.15, .35, .35, .15], [.35, .15, .15, .35]])
lp, lv, c = forward_viterbi("GCGCATTAT", A, E, np.array([.5, .5]))
print(f"{lp:.2f} {lv:.2f}", "".join("HL"[k] for k in c))
# -17.58 -19.98 HHHHLLLLL

```

4.3.4 Baum-Welch: aprender los parámetros

Hasta ahora supusimos conocidos A y E . Si dispusiéramos de secuencias con los estados anotados, estimarlos sería contar: la fracción de veces que H va seguido de L , la fracción de G emitidas por H , etc. Pero, por definición, los estados están ocultos. El algoritmo de Baum et al. (1970) resuelve el problema sustituyendo los conteos que no podemos observar por sus *valores esperados* bajo los parámetros actuales, y repitiendo.

Para cada posición i definimos la probabilidad posterior de una transición concreta y de un estado concreto:

$$\xi_i(k, l) = \mathbb{P}(\pi_i = k, \pi_{i+1} = l \mid x, \theta) = \frac{f_k(i) a_{kl} e_l(x_{i+1}) b_l(i+1)}{\mathbb{P}(x \mid \theta)}, \quad (4.22)$$

$$\gamma_i(k) = \mathbb{P}(\pi_i = k \mid x, \theta) = \frac{f_k(i) b_k(i)}{\mathbb{P}(x \mid \theta)}. \quad (4.23)$$

DONDE

$\xi_i(k, l)$	Probabilidad posterior de que la transición $k \rightarrow l$ ocurra entre las posiciones i e $i + 1$.
$\gamma_i(k)$	Probabilidad posterior de estar en k en la posición i (ecuación (4.19)).

Los conteos esperados se obtienen sumando sobre las posiciones (y sobre las secuencias de entrenamiento, si hay varias), y los nuevos parámetros son las frecuencias relativas de esos conteos:

$$\hat{a}_{kl} = \frac{\sum_{i=1}^{L-1} \xi_i(k, l)}{\sum_{l'} \sum_{i=1}^{L-1} \xi_i(k, l')}, \quad \hat{e}_k(b) = \frac{\sum_{i: x_i=b} \gamma_i(k)}{\sum_{i=1}^L \gamma_i(k)}, \quad \hat{\pi}_k = \gamma_1(k). \quad (4.24)$$

DONDE

$\hat{a}_{kl}, \hat{e}_k(b), \hat{\pi}_k$	Parámetros reestimados para la siguiente iteración.
$\sum_{i: x_i=b}$	Suma solo sobre las posiciones en las que se observó el símbolo b .

Se calculan forward y backward con los parámetros actuales (paso E), se reestiman los parámetros con la ecuación (4.24) (paso M) y se itera hasta que la verosimilitud deja de aumentar. Baum-Welch es un caso particular del algoritmo EM, formulado en general años después por Dempster et al. (1977), y hereda su propiedad fundamental.

Teorema 4.4. (Monotonía de Baum-Welch)

Cada iteración de la ecuación (4.24) cumple $P(x | \hat{\theta}) \geq P(x | \theta)$, con igualdad solo en un punto estacionario de la verosimilitud (Baum et al., 1970).

El teorema garantiza la convergencia a un máximo *local*, no al global. La verosimilitud de un HMM tiene en general muchos máximos locales, así que en la práctica se ejecuta el algoritmo desde varios puntos de partida, o se inicializa con una estimación razonable (por ejemplo, a partir de un MSA, como veremos con los perfiles).

4.3.5 HMM de pares: el alineamiento como modelo probabilístico

Estamos ahora en condiciones de cerrar un cabo que dejamos suelto en el ???. Las tres matrices de Gotoh (??) corresponden a tres estados de un HMM que, en lugar de emitir una secuencia, emite *dos* a la vez: un estado M emite un par alineado

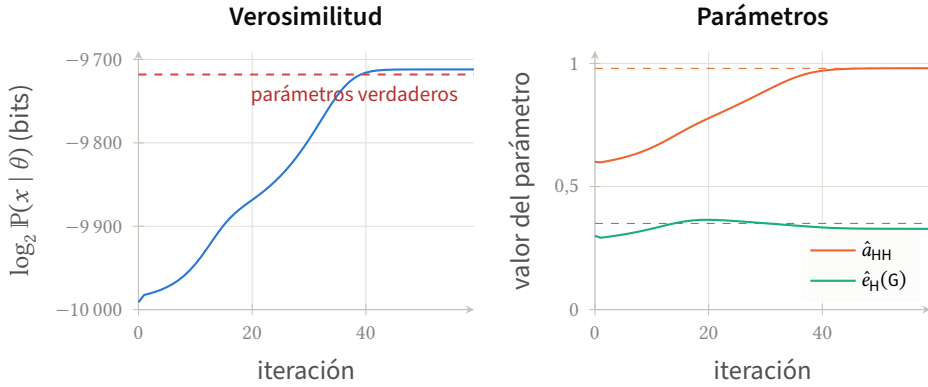


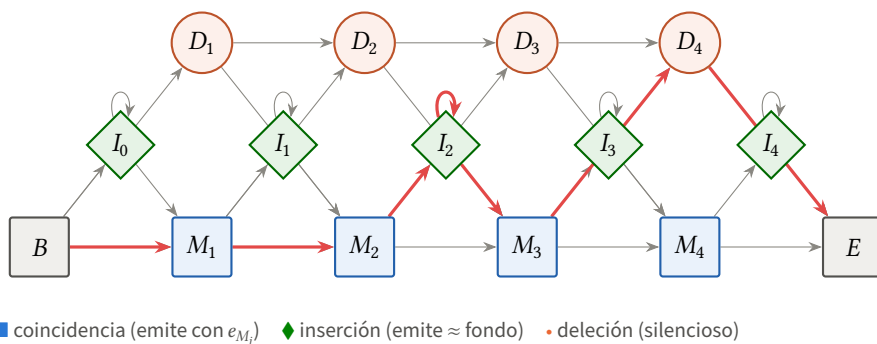
Figura 4.12. Baum-Welch aplicado a 5000 bases simuladas con $a_{HH} = a_{LL} = 0,98$, partiendo de parámetros deliberadamente erróneos ($a_{HH} = 0,6$; emisiones casi uniformes). **Izquierda:** la log-verosimilitud aumenta en cada iteración, como garantiza el teorema 4.4, desde -9991 hasta -9712 bits; tras 47 iteraciones queda a menos de 0,1 bits de su valor final. Termina *por encima* de la de los parámetros verdaderos (línea roja, -9718): el modelo se ajusta a las fluctuaciones de la muestra. **Derecha:** los parámetros convergen a $\hat{a}_{HH} = 0,981$ y $\hat{e}_H(G) = 0,328$, cerca de los valores verdaderos (discontinuas).

(x_i, y_j) con probabilidad $p_{x_i y_j}$; un estado X emite x_i frente a un hueco; un estado Y emite y_j frente a un hueco. La transición $M \rightarrow X$ tiene probabilidad δ (abrir un hueco) y $X \rightarrow X$, probabilidad ε (extenderlo). Durbin et al. (1998) muestran que el algoritmo de Viterbi de este HMM de pares, escrito en log-razones frente a un modelo de fondo, es el algoritmo de Gotoh, con

$$s(a, b) = \log \frac{p_{ab}}{q_a q_b} + \text{cte.}, \quad d_o \approx -\log \delta, \quad d_e \approx -\log \varepsilon.$$

4.3.6 HMM de perfil: un modelo para cada familia

Un perfil (ecuación (4.6)) asigna una distribución propia a cada columna de un MSA; una PWM hace lo mismo para motivos sin huecos. Lo que les falta a ambos es una forma principiada de modelar inserciones y deleciones que dependan de la posición: en una familia de proteínas, los bucles superficiales aceptan inserciones con facilidad, mientras que el núcleo hidrofóbico casi nunca las tolera. Krogh et al. (1994) resolvieron el problema con una arquitectura de HMM que hoy llamamos *HMM de perfil*, y Eddy (1998) la convirtió en una herramienta de uso general.



camino de CAACT -: $B M_1 M_2 I_2 I_3 M_3 D_4 E$

Figura 4.13. Arquitectura de un HMM de perfil de longitud 4, en la variante *Plan7* de HMMER, que no permite transiciones directas entre estados de inserción y delección. Los estados M_j (cuadrados) y D_j (círculos) forman la columna vertebral; los I_j (rombos) cuelgan entre posiciones y tienen un bucle que permite inserciones de cualquier longitud. En rojo, el camino que recorre la secuencia CAACT del ejemplo 4.6: dos residuos insertados tras la posición 2 y una delección de la posición 4. Toda secuencia alineada al MSA corresponde a un camino, y viceversa.

Definición 4.3. (HMM de perfil)

Un HMM de perfil de longitud m tiene, para cada posición consenso $j = 1, \dots, m$, tres estados: un estado de *coincidencia* M_j , que emite un residuo con distribución $e_{M_j}(\cdot)$ específica de la posición; un estado de *inserción* I_j , que emite residuos extra entre las posiciones j y $j + 1$ con una distribución cercana al fondo; y un estado *silencioso* de *delección* D_j , que salta la posición j sin emitir. Estados inicial (B) y final (E) completan el modelo.

La idea clave es que las penalizaciones de hueco, que en Needleman-Wunsch eran constantes, se convierten en probabilidades de transición *distintas en cada posición*: $a_{M_j D_{j+1}}$ puede ser minúscula en una hélice del núcleo y grande en un bucle. Y como el modelo es un HMM, todo lo que sabemos (forward, Viterbi, Baum-Welch) se aplica directamente.

Construcción a partir de un MSA. Dado un MSA de la familia, se decide primero qué columnas son *consenso*: una regla sencilla es declarar de coincidencia toda columna en la que al menos la mitad de las secuencias tienen un residuo, y de inserción las

demás. Después, cada secuencia del MSA define un camino por el modelo y basta con contar transiciones y emisiones, añadir pseudoconteos (o una previa de mezcla de Dirichlet, Sjölander et al., 1996) y normalizar:

$$e_{M_j}(a) = \frac{c_j(a) + \alpha_a}{\sum_{a'} (c_j(a') + \alpha_{a'})}, \quad a_{kl} = \frac{C_{kl} + \beta}{\sum_{l'} (C_{kl'} + \beta)}. \quad (4.25)$$

DONDE

$c_j(a)$	Número ponderado de secuencias con el residuo a en la columna consenso j .
C_{kl}	Número ponderado de veces que los caminos del MSA usan la transición $k \rightarrow l$.
α_a, β	Pseudoconteos de emisión y de transición.

Los conteos se ponderan (por ejemplo, con los pesos de la ecuación (4.2)) para que las subfamilias sobrerrepresentadas no dominen el modelo.

Ejemplo 4.6. (Un HMM de perfil de juguete)

Consideremos el MSA de cinco secuencias de ADN y seis columnas:

columna	1	2	3	4	5	6
s1	C	A	-	-	T	G
s2	C	A	A	-	T	G
s3	C	G	-	-	T	G
s4	C	A	A	C	T	-
s5	T	A	-	-	T	G
estado	M_1	M_2	I_2	I_2	M_3	M_4

Las columnas 3 y 4 tienen residuos en solo 2 y 1 de las 5 secuencias, así que son de inserción; las demás definen $M_1, \dots, M_4$. Con un pseudoconteo de 1 por base, las emisiones de M_1 (conteos C:4, T:1) son $e_{M_1} = (\frac{1}{9}, \frac{5}{9}, \frac{1}{9}, \frac{2}{9}) = (0,111; 0,556; 0,111; 0,222)$ para (A,C,G,T); la columna 5, invariante, da $e_{M_3}(T) = \frac{5+1}{5+4} = 0,667$; y M_4 , con solo cuatro residuos, $e_{M_4}(G) = \frac{4+1}{4+4} = 0,625$. Los caminos son $B M_1 M_2 M_3 M_4 E$ para s1, s3 y s5; $B M_1 M_2 I_2 M_3 M_4 E$ para s2; y $B M_1 M_2 I_2 M_3 D_4 E$ para s4 (figura 4.13). Desde M_2 hay tres transiciones a M_3 y dos a I_2 ; con $\beta = 1$ sobre las tres salidas posibles (M_3, I_2, D_3), $a_{M_2 M_3} = \frac{4}{8} = 0,5$, $a_{M_2 I_2} = \frac{3}{8} = 0,375$ y $a_{M_2 D_3} = \frac{1}{8} = 0,125$.

Alinear una secuencia al perfil. Para puntuar una secuencia x contra el modelo se usa Viterbi (o forward) en log-razones frente al fondo q . Las recurrencias tienen la misma forma que las de Gotoh, pero con parámetros que dependen de j :

$$V_j^M(i) = \log \frac{e_{M_j}(x_i)}{q_{x_i}} + \max \begin{cases} V_{j-1}^M(i-1) + \log a_{M_{j-1}M_j} \\ V_{j-1}^I(i-1) + \log a_{I_{j-1}M_j} \\ V_{j-1}^D(i-1) + \log a_{D_{j-1}M_j} \end{cases} \quad (4.26)$$

$$V_j^I(i) = \log \frac{e_{I_j}(x_i)}{q_{x_i}} + \max \begin{cases} V_j^M(i-1) + \log a_{M_jI_j} \\ V_j^I(i-1) + \log a_{I_jI_j} \end{cases} \quad (4.27)$$

$$V_j^D(i) = \max \begin{cases} V_{j-1}^M(i) + \log a_{M_{j-1}D_j} \\ V_{j-1}^D(i) + \log a_{D_{j-1}D_j} \end{cases} \quad (4.28)$$

DONDE

$V_j^M(i)$	Mejor log-razón de un camino que ha emitido $x_1 \dots x_i$ y termina en M_j (ídem para I_j, D_j).
q_{x_i}	Frecuencia de fondo del residuo x_i ; normalmente $e_{I_j} \approx q$, por lo que el primer término de la ecuación (4.27) es casi cero.
i, j	Posición en la secuencia y posición en el modelo, respectivamente.

Obsérvese que D_j no consume residuos (su índice i no cambia) y que, siguiendo la arquitectura Plan7, no hay transiciones $I \leftrightarrow D$. El costo es $O(mL)$, el mismo que alinear dos secuencias de longitudes m y L . Sustituyendo cada $\max$ por una suma (en espacio de probabilidades) se obtiene el algoritmo forward del perfil, que da la probabilidad de la secuencia bajo el modelo sumando sobre *todos* sus alineamientos posibles.

4.3.7 HMMER y Pfam

HMMER implementa este programa completo. Sus versiones iniciales puntuaban con Viterbi y eran unas cien veces más lentas que BLAST, lo que limitaba su uso. HMMER3 (Eddy, 2011) cambió la situación con dos ideas: puntuar con el algoritmo forward completo, que es más sensible porque integra la evidencia de todos los alineamientos, y colocar delante una cascada de filtros heurísticos que descartan pronto la inmensa mayoría de las secuencias no homólogas (figura 4.14). El primero, el filtro MSV (*multiple segment Viterbi*), calcula la mejor suma de varios segmentos sin huecos con instrucciones vectoriales SIMD, la misma técnica que acelera Smith-

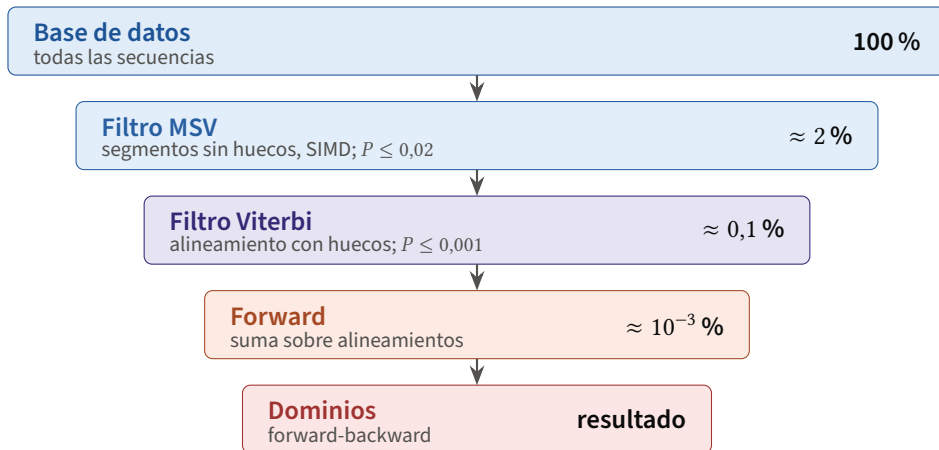


Figura 4.14. La cascada de aceleración de HMMER3 (Eddy, 2011). Cada etapa es más lenta y más sensible que la anterior, pero solo procesa las secuencias que la anterior dejó pasar. Los umbrales se expresan como valores P : el filtro MSV deja pasar, por defecto, el 2 % de las secuencias no homólogas, y el de Viterbi, el 0,1 %. La puntuación final, sobre la que se calculan los valores E , es siempre la del algoritmo forward completo; los filtros solo deciden a quién merece la pena aplicarlo.

Waterman. Según el propio artículo, HMMER3 es entre 100 y 1000 veces más rápido que HMMER2 y aproximadamente tan rápido como BLAST para proteínas.

HMMER expresa sus resultados igual que BLAST en el ??: una puntuación en bits (log-razón en base 2 entre el modelo y el fondo) y un valor E . Para asignar valores E sin simulaciones costosas, Eddy (2011) conjeturó, y comprobó empíricamente con miles de modelos, que las puntuaciones Viterbi en bits de secuencias no homólogas siguen una distribución de Gumbel de pendiente fija $\lambda = \ln 2$, y que la cola alta de las puntuaciones forward sigue una exponencial de la misma pendiente. Así, solo hay que estimar un parámetro de posición por modelo, lo que se hace una vez al construirlo.

Construir un HMM de perfil y buscar homólogos

```
# 1. HMM a partir de un MSA (Stockholm o FASTA alineado)
hmmbuild globinas.hmm globinas.sto
# 2. Buscar el perfil en una base de datos de proteínas
hmmsearch --tblout aciertos.tbl -E 1e-5 globinas.hmm uniprot_sprot.fasta
# 3. Anotar dominios de una proteína contra todo Pfam
```

```
hmmcompress Pfam-A.hmm # indexa la base (una vez)
hmmsearch --domtblout dominios.tbl Pfam-A.hmm consulta.fasta
```

```
leer_hmmer.py
```

```
from Bio import SearchIO

for q in SearchIO.parse("dominios.tbl", "hmmsearch3-domtbl"):
    for hit in q:
        for hsp in hit:
            print(f"{hit.id:15s} {hsp.bitscore:6.1f} bits "
                  f"E={hsp.evalue:.1e} "
                  f"{hsp.query_start + 1}-{hsp.query_end}")
```

La aplicación más influyente de los HMM de perfil es la anotación de dominios. Pfam (Mistry et al., 2021) es una colección de familias de proteínas, cada una definida por un *alineamiento semilla* curado a mano, un HMM de perfil construido con HMMER a partir de él y un *umbral de recolección* (*gathering threshold*) en bits, elegido por los curadores para que no incluya falsos positivos conocidos. El alineamiento completo de la familia se obtiene buscando el HMM en UniProt. La versión de 2021 añadió más de 350 familias nuevas, revisó las que cubren el proteoma del SARS-CoV-2 y reintrodujo Pfam-B, un suplemento de 136 730 agrupaciones automáticas de secuencias aún no cubiertas por ninguna familia.

De los ribosomas a las familias de proteínas

Los HMM llegaron a la biología desde el reconocimiento del habla, donde el tutorial de Rabiner (1989) los había popularizado. El grupo de David Haussler, en Santa Cruz, los adaptó a las proteínas: Krogh et al. (1994) modelaron con la arquitectura de la figura 4.13 las globinas, las cinasas y las proteínas EF-hand, y mostraron que un modelo entrenado solo con secuencias no alineadas podía producir un MSA y detectar miembros lejanos de la familia. El libro de Durbin et al. (1998), escrito por cuatro protagonistas de esta historia, sigue siendo la mejor introducción a todo el tema.

PWM: una distribución por posición, sin huecos.
HMM de perfil: una distribución por posición, con huecos que también dependen de la posición.

En resumen

- Un MSA se puntúa típicamente con la suma de pares, idealmente ponderada para compensar secuencias redundantes. Su optimización exacta cuesta $O(k^2 2^k n^k)$ y es NP-difícil.
- El alineamiento progresivo (distancias, árbol guía, alineamiento de perfiles) es rápido pero codicioso: “una vez un hueco, siempre un hueco”. El refinamiento iterativo (MUSCLE), la consistencia (T-Coffee), la FFT (MAFFT) y los árboles mBed con alineamiento HMM–HMM (Clustal Omega) mitigan sus errores o lo escalan.
- Una PWM resume sitios alineados como log-razones $\log_2(f_{b,i}/q_b)$; los pseudo-conteos evitan probabilidades nulas. Su puntuación esperada en sitios verdaderos es la entropía relativa total del motivo.
- El contenido de información $R_i = 2 - H_i$ mide la conservación de cada columna en bits; el *sequence logo* lo muestra apilando letras de altura $f_{b,i}R_i$.
- Un HMM se evalúa con forward, se decodifica con Viterbi o por probabilidades posteriores y se entrena con Baum-Welch (EM), todos en $O(LK^2)$.
- Un HMM de perfil añade a cada posición estados de coincidencia, inserción y delección, con parámetros propios. HMMER3 lo busca en bases de datos con filtros acelerados y puntuación forward; Pfam lo usa para definir y anotar familias de dominios.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 4.1 ●●● Calcule a mano la puntuación SP (BLOSUM62, $d = 8$) de la columna K, K, R, -, E. ¿Cuánto cambia si se asigna peso 0,5 a cada una de las dos primeras secuencias y peso 1 a las demás?
- 4.2 ●●● Escriba la función `suma_de_pares(filas, d)` que implementa la ecuación (4.1) con penalización lineal y compruebe que las cuatro primeras columnas del ejemplo 4.1 suman 62.
- 4.3 ●●● Una columna de un motivo tiene frecuencias (0,5; 0,5; 0; 0) y otra (0,7; 0,1; 0,1; 0,1). ¿Cuál tiene más contenido de información? Dibuje a mano sus pilas en un *logo*.

- 4.4 ●●● Con la matriz de distancias de la figura 4.2, construya el árbol por *neighbor-joining* en lugar de UPGMA. ¿Cambia el orden en que se alinean los perfiles? Alinee las globinas con Clustal Omega y MAFFT y cuente las columnas en que difieren.
- 4.5 ●●● Descargue de JASPAR la matriz de conteos de otro factor (por ejemplo, CTCF, MA0139) y calcule su contenido total de información. ¿Cada cuántas bases esperaría un sitio por azar? Compare con el número de sitios de CTCF que se conocen en el genoma humano.
- 4.6 ●●● Verifique numéricamente, con el código de `forward_viterbi.py`, que $\sum_k f_k(i) b_k(i) = P(x)$ para *todas* las posiciones i . ¿Por qué debe cumplirse esta identidad?
- 4.7 ●●● Implemente Baum-Welch con reescalado para el modelo de dos estados y reproduzca la figura 4.12. Repita desde 20 puntos de partida aleatorios: ¿llegan todos al mismo máximo? ¿Qué ocurre si inicializa con $e_H = e_L$ exactamente?
- 4.8 ●●● Construya con `hmmbuild` un HMM de perfil a partir del MSA de globinas y búsquelo con `hmmsearch` en Swiss-Prot. Compare el número de aciertos con $E < 10^{-5}$ con el que obtiene `blastp` usando como consulta solo la hemoglobina β humana. Explique la diferencia en términos de la ecuación (4.26).

Referencias del capítulo

- Baum, L. E., Petrie, T., Soules, G., & Weiss, N. (1970). A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains. *The Annals of Mathematical Statistics*, 41(1), 164-171. <https://doi.org/10.1214/aoms/1177697196>
- Carrillo, H., & Lipman, D. (1988). The multiple sequence alignment problem in biology. *SIAM Journal on Applied Mathematics*, 48(5), 1073-1082. <https://doi.org/10.1137/0148063>
- Castro-Mondragon, J. A., Riudavets-Puig, R., Rauluseviciute, I., Berhanu Lemma, R., Turchi, L., Blanc-Mathieu, R., Lucas, J., Boddie, P., Khan, A., Manosalva Pérez, N., Fornes, O., Leung, T. Y., Aguirre, A., Hammal, F., Schmelter, D., Baranasic, D., Ballester, B., Sandelin, A., Lenhard, B., ... Mathelier, A. (2022). JASPAR 2022: The 9th release of the open-access database of transcription factor binding profiles. *Nucleic Acids Research*, 50(D1), D165-D173. <https://doi.org/10.1093/nar/gkab1113>

- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: Freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Crooks, G. E., Hon, G., Chandonia, J.-M., & Brenner, S. E. (2004). WebLogo: A sequence logo generator. *Genome Research*, 14(6), 1188-1190. <https://doi.org/10.1101/gr.849004>
- Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1), 1-22. <https://doi.org/10.1111/j.2517-6161.1977.tb01600.x>
- Durbin, R., Eddy, S. R., Krogh, A., & Mitchison, G. (1998). *Biological sequence analysis: Probabilistic models of proteins and nucleic acids*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511790492>
- Eddy, S. R. (1998). Profile hidden Markov models. *Bioinformatics*, 14(9), 755-763. <https://doi.org/10.1093/bioinformatics/14.9.755>
- Eddy, S. R. (2004). What is a hidden Markov model? *Nature Biotechnology*, 22(10), 1315-1316. <https://doi.org/10.1038/nbt1004-1315>
- Eddy, S. R. (2011). Accelerated profile HMM searches. *PLoS Computational Biology*, 7(10), Artículo e1002195. <https://doi.org/10.1371/journal.pcbi.1002195>
- Edgar, R. C. (2004). MUSCLE: Multiple sequence alignment with high accuracy and high throughput. *Nucleic Acids Research*, 32(5), 1792-1797. <https://doi.org/10.1093/nar/gkh340>
- Edgar, R. C., & Batzoglou, S. (2006). Multiple sequence alignment. *Current Opinion in Structural Biology*, 16(3), 368-373. <https://doi.org/10.1016/j.sbi.2006.04.004>
- Feng, D.-F., & Doolittle, R. F. (1987). Progressive sequence alignment as a prerequisite to correct phylogenetic trees. *Journal of Molecular Evolution*, 25(4), 351-360. <https://doi.org/10.1007/BF02603120>
- Henikoff, S., & Henikoff, J. G. (1994). Position-based sequence weights. *Journal of Molecular Biology*, 243(4), 574-578. [https://doi.org/10.1016/0022-2836\(94\)90032-9](https://doi.org/10.1016/0022-2836(94)90032-9)
- Higgins, D. G., & Sharp, P. M. (1988). CLUSTAL: A package for performing multiple sequence alignment on a microcomputer. *Gene*, 73(1), 237-244. [https://doi.org/10.1016/0378-1119\(88\)90330-7](https://doi.org/10.1016/0378-1119(88)90330-7)
- Katoh, K., Misawa, K., Kuma, K.-i., & Miyata, T. (2002). MAFFT: A novel method for rapid multiple sequence alignment based on fast Fourier transform. *Nucleic Acids Research*, 30(14), 3059-3066. <https://doi.org/10.1093/nar/gkf436>
- Krogh, A., Brown, M., Mian, I. S., Sjölander, K., & Haussler, D. (1994). Hidden Markov models in computational biology: Applications to protein modeling. *Journal of Molecular Biology*, 235(5), 1501-1531. <https://doi.org/10.1006/jmbi.1994.1104>

- Lawrence, C. E., Altschul, S. F., Boguski, M. S., Liu, J. S., Neuwald, A. F., & Wootton, J. C. (1993). Detecting subtle sequence signals: A Gibbs sampling strategy for multiple alignment. *Science*, 262(5131), 208-214. <https://doi.org/10.1126/science.8211139>
- Mistry, J., Chuguransky, S., Williams, L., Qureshi, M., Salazar, G. A., Sonnhammer, E. L. L., Tosatto, S. C. E., Paladin, L., Raj, S., Richardson, L. J., Finn, R. D., & Bateman, A. (2021). Pfam: The protein families database in 2021. *Nucleic Acids Research*, 49(D1), D412-D419. <https://doi.org/10.1093/nar/gkaa913>
- Notredame, C., Higgins, D. G., & Heringa, J. (2000). T-Coffee: A novel method for fast and accurate multiple sequence alignment. *Journal of Molecular Biology*, 302(1), 205-217. <https://doi.org/10.1006/jmbi.2000.4042>
- Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2), 257-286. <https://doi.org/10.1109/5.18626>
- Schneider, T. D., & Stephens, R. M. (1990). Sequence logos: A new way to display consensus sequences. *Nucleic Acids Research*, 18(20), 6097-6100. <https://doi.org/10.1093/nar/18.20.6097>
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- Sievers, F., Wilm, A., Dineen, D., Gibson, T. J., Karplus, K., Li, W., Lopez, R., McWilliam, H., Remmert, M., Söding, J., Thompson, J. D., & Higgins, D. G. (2011). Fast, scalable generation of high-quality protein multiple sequence alignments using Clustal Omega. *Molecular Systems Biology*, 7(1), Artículo 539. <https://doi.org/10.1038/msb.2011.75>
- Sjölander, K., Karplus, K., Brown, M., Hughey, R., Krogh, A., Mian, I. S., & Haussler, D. (1996). Dirichlet mixtures: A method for improved detection of weak but significant protein sequence homology. *Bioinformatics*, 12(4), 327-345. <https://doi.org/10.1093/bioinformatics/12.4.327>
- Stormo, G. D. (2000). DNA binding sites: Representation and discovery. *Bioinformatics*, 16(1), 16-23. <https://doi.org/10.1093/bioinformatics/16.1.16>
- Stormo, G. D., Schneider, T. D., Gold, L., & Ehrenfeucht, A. (1982). Use of the «Perceptron» algorithm to distinguish translational initiation sites in *E. coli*. *Nucleic Acids Research*, 10(9), 2997-3011. <https://doi.org/10.1093/nar/10.9.2997>
- Thompson, J. D., Higgins, D. G., & Gibson, T. J. (1994). CLUSTAL W: Improving the sensitivity of progressive multiple sequence alignment through sequence weighting, position-specific gap penalties and weight matrix choice. *Nucleic Acids Research*, 22(22), 4673-4680. <https://doi.org/10.1093/nar/22.22.4673>
- Viterbi, A. J. (1967). Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2), 260-269. <https://doi.org/10.1109/TIT.1967.1054010>
- Wang, L., & Jiang, T. (1994). On the complexity of multiple sequence alignment. *Journal of Computational Biology*, 1(4), 337-348. <https://doi.org/10.1089/cmb.1994.1.337>